



Introduction to GitHub Packages

GitHub Packages is a software package hosting service that allows you to host your software packages privately or publicly and use packages as dependencies in your projects.

[Copy as Markdown](#) ▾

In this article

[About GitHub Packages](#)[About linked artifacts for organizations](#)[About billing for GitHub Packages](#)[Supported clients and formats](#)[Authenticating to GitHub Packages](#)[Managing packages](#)[Contacting support](#)

About GitHub Packages

GitHub Packages is a platform for hosting and managing packages, including containers and other dependencies. GitHub Packages combines your source code and packages in one place to provide integrated permissions management and billing, so you can centralize your software development on GitHub.

You can integrate GitHub Packages with GitHub's APIs, GitHub Actions, and webhooks to create an end-to-end DevOps workflow that includes your code, CI, and deployment solutions.

GitHub Packages offers different package registries for commonly used package managers, such as npm, RubyGems, Apache Maven, Gradle, Docker, and NuGet. GitHub's Container registry is optimized for containers and supports Docker and OCI images. For more information on the different package registries that GitHub Packages supports, see [Working with a GitHub Packages registry](#).

You can view a package's README, as well as metadata such as licensing, download statistics, version history, and more on GitHub. For more information, see [Viewing packages](#).

Overview of package permissions

The permissions for a package are either inherited from the repository where the package is hosted, or can be defined for specific users or organizations. Some registries only support permissions inherited from a repository. For a list of these registries, see [About permissions for GitHub Packages](#). For more information on package access, see [Configuring a package's access control and visibility](#).

Overview of package visibility

You can publish packages in a public repository (public packages) to share with all of GitHub, or in a private repository (private packages) to share with collaborators or an organization.

About linked artifacts for organizations

The linked artifacts page is an alternative view that you can also access in the "Packages" section of an organization's settings.

Like GitHub Packages, the linked artifacts page allows you to collect information about your organization's builds in a single place. Teams can use the linked artifacts page to find an artifact's source code, build details, and deployment history.

Unlike GitHub Packages, the linked artifacts page does **not** host the package or image files themselves. Instead, it provides an authoritative source for the metadata associated with each package or image.

Your organization may benefit from using the linked artifacts page either:

- **Alongside** GitHub Packages, as a complementary view focused on the compliance and security aspects of package consumption
- **As an alternative to** GitHub Packages, allowing you to store your packages on an external registry of your choice while maintaining visibility of the packages on GitHub

For more information, see [About linked artifacts](#).

About billing for GitHub Packages

GitHub Packages usage is **free** for **public packages**.

For **private packages**, each account on GitHub receives an amount of **free storage and data transfer**, determined by the account's plan. Any usage beyond the included amounts is controlled by budgets.

If your account does not have a valid payment method on file, usage is blocked once you use up your quota.

If you have a valid payment method on file, spending may be limited by one or more budgets. Check the budgets set for your account to ensure they are appropriate for your usage needs. See [Setting up budgets to control spending on metered products](#).

For more information, see [GitHub Packages billing](#).

Supported clients and formats

GitHub Packages uses the native package tooling commands you're already familiar with to publish and install package versions.

Support for package registries

Language	Description	Package format	Package client
JavaScript	Node package manager	<code>package.json</code>	<code>npm</code>
Ruby	RubyGems package manager	<code>Gemfile</code>	<code>gem</code>
Java	Apache Maven project management and comprehension tool	<code>pom.xml</code>	<code>mvn</code>
Java	Gradle build automation tool for Java	<code>build.gradle</code> or <code>build.gradle.kts</code>	<code>gradle</code>

Language	Description	Package format	Package client
.NET	NuGet package management for .NET	nupkg	dotnet CLI
N/A	Docker container management	Dockerfile	Docker

For more information about configuring your package client for use with GitHub Packages, see [Working with a GitHub Packages registry](#).

For more information about Docker and the Container registry, see [Working with the Container registry](#).

Authenticating to GitHub Packages

Note

GitHub Packages only supports authentication using a personal access token (classic). For more information, see [Managing your personal access tokens](#).

You need an access token to publish, install, and delete private, internal, and public packages.

You can use a personal access token (classic) to authenticate to GitHub Packages or the GitHub API. When you create a personal access token (classic), you can assign the token different scopes depending on your needs. For more information about packages-related scopes for a personal access token (classic), see [About permissions for GitHub Packages](#).

To authenticate to a GitHub Packages registry within a GitHub Actions workflow, you can use:

- `GITHUB_TOKEN` to publish packages associated with the workflow repository.
- A personal access token (classic) with at least `read:packages` scope to install packages associated with other private repositories (`GITHUB_TOKEN` can be used if the repository is granted read access to the package. See [Configuring a package's access control and visibility](#)).

For more information about `GITHUB_TOKEN` used in GitHub Actions workflows, see [Use GITHUB_TOKEN for authentication in workflows](#).

Managing packages

You can delete a package in the GitHub user interface or using the REST API. For more information, see [Deleting and restoring a package](#) and the [REST API endpoints for packages](#). For certain registries, you can use GraphQL to delete a version of a private package.

You cannot use the GitHub Packages GraphQL API with registries that support granular permissions. For the registries that **only** support repository-scoped permissions, and can be used with the GraphQL API, see [About permissions for GitHub Packages](#).

When you use the GraphQL API to query and delete private packages, you must use the same personal access token (classic) you use to authenticate to GitHub Packages.

For more information, see [Forming calls with GraphQL](#).

You can configure webhooks to subscribe to package-related events, such as when a package is published or updated. For more information, see the [Webhook events and payloads](#).

Contacting support

If you have feedback or feature requests for GitHub Packages, use a [GitHub Community discussion](#).

Contact us through the [GitHub Support portal](#) about GitHub Packages if:

- You experience anything that contradicts the documentation
- You encounter vague or unclear errors
- Your published package contains sensitive data, such as GDPR violations, API Keys, or personally identifying information

Legal

© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Status](#) [Pricing](#) [Expert services](#) [Blog](#)